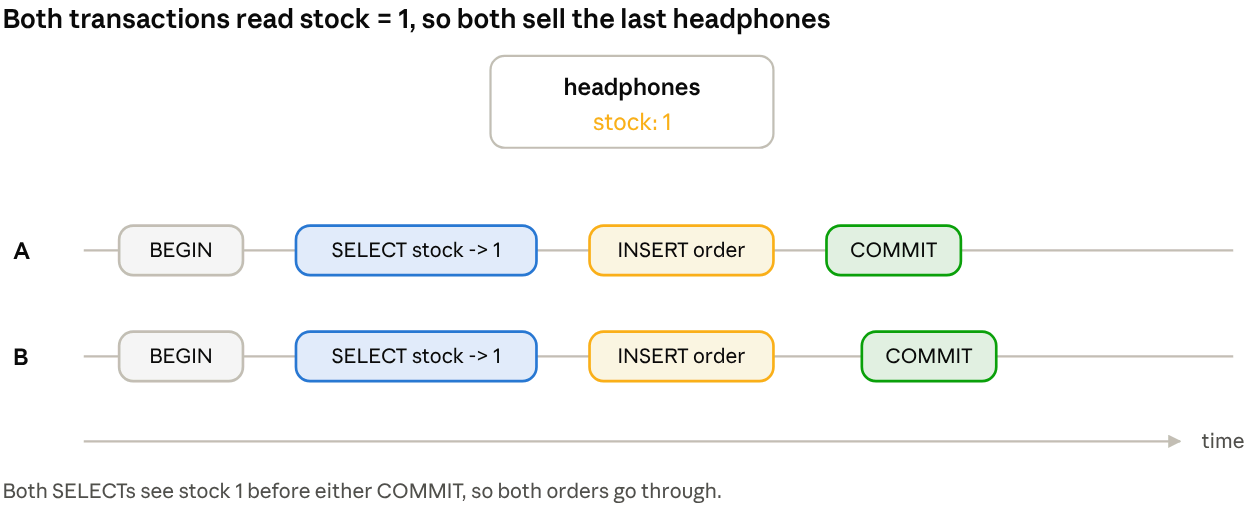
System Design Notes: Database Isolation

Nov 1, 2026 · @Salike Hassan

Why this matters

Think of an e-commerce site. There is one pair of headphones left (stock = 1) and two buyers, A and B, try to buy it at the same moment. Each one starts a transaction (`BEGIN`), reads the stock, inserts an order and commits.

Both read stock = 1, both insert an order, both commit. We have sold the same headphones twice, even though the stock was only 1.

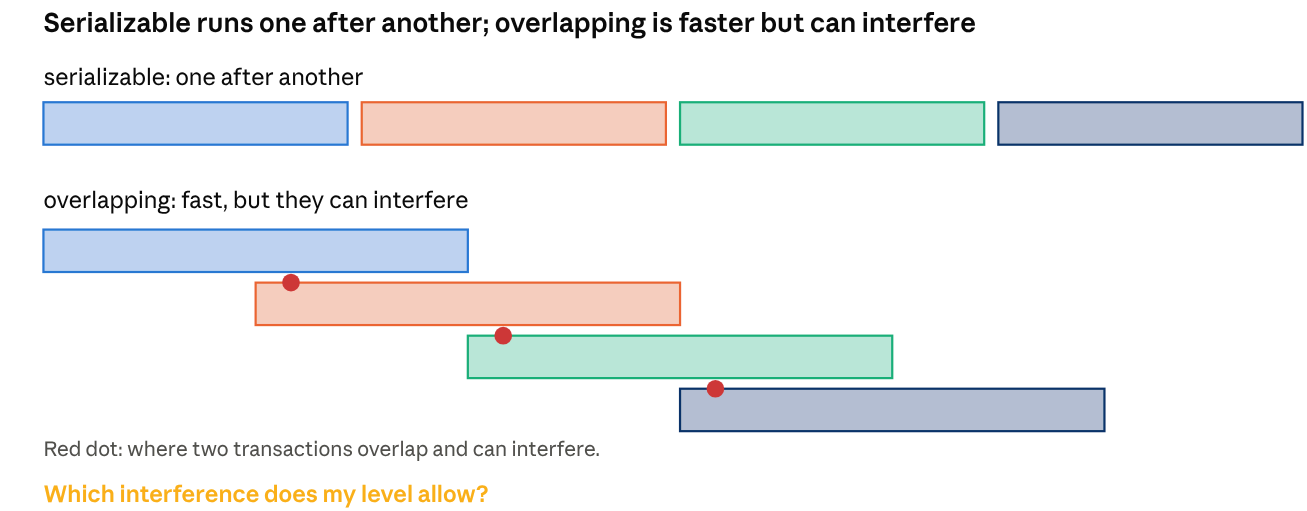


headphones race · transactions A and B

What isolation means

The **I in ACID**. Every transaction should behave as if it were the only one running, and the others run one after another. The name for that ideal is **serializable**.

Running them truly one after another would kill the throughput. So databases let transactions **overlap**, and give us levels that say how much interference we accept. The question to ask is: **which interference does my level allow?**

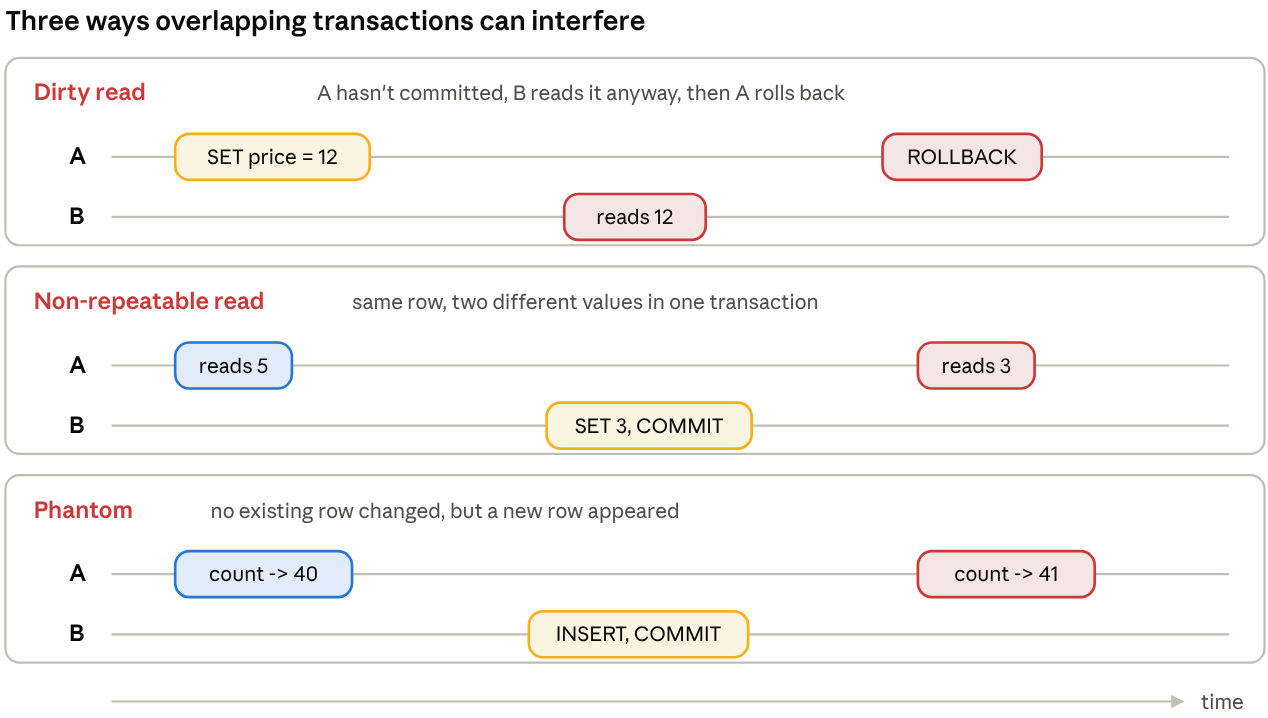


serial vs overlapping transactions

The three anomalies

Interference between overlapping transactions comes in three classic forms.

1. **Dirty read:** A sets a price to 12 but hasn't committed. B reads 12. Then A rolls back. B acted on a value that never officially existed.
2. **Non-repeatable read:** A reads the stock and sees 5. B sets it to 3 and commits. A reads again, in the same transaction, and sees 3. The **same row** gave two answers.
3. **Phantom:** A counts today's orders and gets 40. B inserts a new order and commits. A counts again and gets 41. No existing row changed, but a **new row appeared** in the set.



dirty read, non-repeatable read, phantom

Quick way to tell them apart: dirty read is about **uncommitted** data, non-repeatable is about a **row that changed**, phantom is about **rows that appeared or disappeared**.

The four isolation levels

The SQL standard defines four levels. Each one prevents more anomalies than the one before.

Level	Dirty read	Non-repeatable read	Phantom
Read uncommitted	possible	possible	possible
Read committed	prevented	possible	possible
Repeatable read	prevented	prevented	possible
Serializable	prevented	prevented	prevented

Higher level = fewer surprises, but more locking or more aborted transactions, so lower throughput.

Not in this table: lost update and write skew. The headphones example is one of these: both transactions check the stock, both act on what they saw. They are covered further down, because the standard table doesn't cover them and the level alone doesn't always stop them.

Isolation levels in real SQL databases

The table above is what the **standard** says. Real engines use the same level names but behave differently, so the name alone doesn't tell you what you get. Most modern engines use **MVCC** (multi-version concurrency control): a reader sees a consistent snapshot of the data and doesn't block writers.

Database	Default level	What to know
PostgreSQL	Read committed	Read uncommitted behaves as read committed, so dirty reads never happen. Repeatable read is snapshot isolation, so it also prevents phantoms, but write skew is still possible. Serializable uses SSI and may abort a transaction with a serialization error, so the app must retry.
MySQL (InnoDB)	Repeatable read	Plain reads use a snapshot taken at the first read. Locking reads (`SELECT ... FOR UPDATE`) use next-key locks to block phantoms. Serializable turns plain selects into shared-lock reads, so it is lock-based.
SQL Server	Read committed	By default read committed uses locks. Turn on read committed snapshot (RCSI) for row versioning, or use the separate snapshot level. Repeatable read and serializable are lock-based. `NOLOCK` allows dirty reads.
Oracle	Read committed	No read uncommitted. Its serializable is snapshot-based, not truly serializable, so write skew is possible.

What this means in practice:

- **Check what your database does**, not what the standard says. Repeatable read in PostgreSQL is not the same as in MySQL.

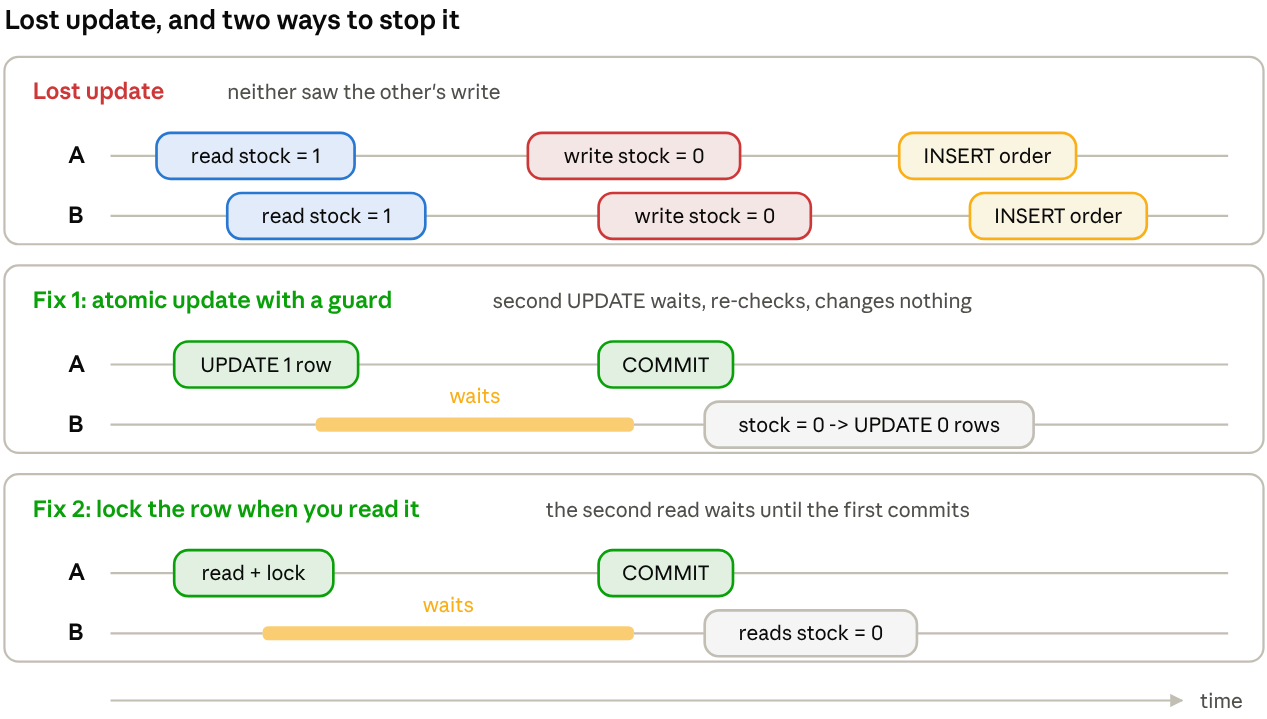
- At **serializable**, be ready to **retry** aborted transactions (PostgreSQL) or to accept more blocking (lock-based engines).
- For the headphones case, don't depend on the level alone. An atomic `UPDATE products SET stock = stock - 1 WHERE id = ? AND stock > 0` (then check the rows affected), or `SELECT ... FOR UPDATE` to lock the row, closes the gap at any level. Details in the lost update section below.

Lost update (the headphones, continued)

Back to the headphones. The oversell comes from a **read-modify-write**: read the stock, change it in the app, write it back.

1. A and B both read stock = 1.
2. Each one works out in the app that the new stock is 0.
3. Each writes stock = 0, then inserts an order.

Neither saw the other's write. We have two orders for one pair of headphones, and the second decrement was lost (stock should have gone to -1, or B should have been refused).



lost update and two fixes · timelines A and B

What SQL Server does at each level

Level	What happens to the read-modify-write
Read committed (default, lock-based)	Reads release their lock straight away, so both read 1 and both write. **Lost update.**
Read committed snapshot (RCSI)	Plain reads don't block and see the last committed value, so both read 1. **Lost update.**
Repeatable read	Reads keep their shared lock until commit. Both then try to UPDATE and each waits for the other. **Deadlock:** one is killed with error 1205.
Snapshot	The second writer's UPDATE fails with error 3960 (update conflict). **Retry.**
Serializable	Same as repeatable read: a deadlock, one victim.

So the level alone doesn't reliably protect a read-modify-write, and each level fails in a different way. Fix it in the query.

Fix 1: atomic update with a guard

Let the database do the arithmetic, with a guard, then check how many rows changed.

```
UPDATE products
SET stock = stock - 1
WHERE id = 7 AND stock > 0;

-- @@ROWCOUNT = 1: we got the last pair
-- @@ROWCOUNT = 0: sold out, refuse the order
```

The second UPDATE waits for the first to commit, re-checks the condition, finds stock at 0 and updates nothing.

Fix 2: lock the row when you read it

SQL Server has no `SELECT ... FOR UPDATE` (that exists only for cursors). The equivalent is the `UPDLOCK` hint.

```
BEGIN TRAN;
SELECT stock FROM products WITH (UPDLOCK, ROWLOCK) WHERE id = 7;
-- change it in the app, then:
UPDATE products SET stock = stock - 1 WHERE id = 7;
COMMIT;
```

The second transaction's locking read waits until the first commits, then reads stock = 0.

Fix 3: version check (optimistic)

Keep a `rowversion` column. Read it with the row and update only if it is unchanged.

```
UPDATE products
SET stock = @newStock
WHERE id = 7 AND row_version = @versionIRead;
-- @@ROWCOUNT = 0: someone changed it first, re-read and retry
```

No lock is held while the app thinks. It suits cases where conflicts are rare.

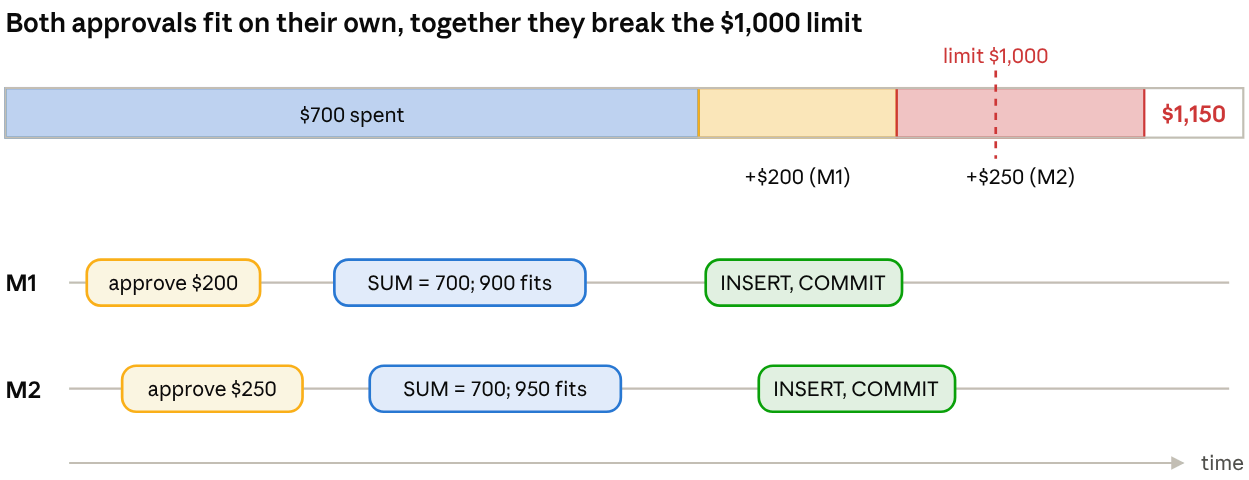
A simple rule of thumb: Fix 1 for counters like stock, Fix 2 when the logic between read and write is complex, Fix 3 when users edit a record and clashes are rare.

Write skew (the budget approval)

Second example, a nastier one. A team has an expense budget of $1,000 and $700 is already spent. The rule: **never approve an expense that pushes the total over the limit.**

1. Manager M1 approves $200. The check reads SUM = 700, so 900 fits.
2. Manager M2 approves $250 at the same time. The check also reads SUM = 700, so 950 fits.
3. Each inserts its expense and commits.

The total is now $1,150. The rule is broken.



Each checks the total before the other commits, then inserts a different new row.

budget approval · write skew

id	team	amount	
301	9	$400	
302	9	$300	
303	9	$200	M1
304	9	$250	M2

This is **write skew**: each transaction checked a *premise* (the total is under the limit), then acted on it, and each wrote a **different new row**. There is no write conflict to detect.

Why the usual tools don't stop it

- **Row locks:** M1 and M2 write different rows, so nothing conflicts.
- **Repeatable read:** it only locks the rows it read (301, 302). The two new rows are phantoms, so they are not covered.
- **Snapshot isolation:** different rows, so no update conflict. Both commit.

Fix: lock the premise

Lock something that **both** transactions touch. Keep a budget row per team and lock it before checking.

```
BEGIN TRAN;
SELECT spent FROM team_budgets WITH (UPDLOCK, ROWLOCK) WHERE team_id = 9;
-- check spent + @amount <= 1000, then:
INSERT INTO expenses (team, amount) VALUES (9, @amount);
UPDATE team_budgets SET spent = spent + @amount WHERE team_id = 9;
COMMIT;
```

Both approvals now queue on the same row, so the second one sees the new total. A single guarded update does the same in one statement: `UPDATE team_budgets SET spent = spent + @amount WHERE team_id = 9 AND spent + @amount <= 1000`.

The other option is serializable isolation, next.

Serializable in SQL Server

In SQL Server, serializable is **lock-based**. The SUM over team 9 takes **key-range locks** on the range it read and keeps them until commit, so nobody can insert into that range.

For the budget example:

1. M1 and M2 both read the range. Shared range locks don't conflict, so both proceed.
2. M1 tries to insert and waits for M2's range lock. M2 tries to insert and waits for M1's. A **deadlock**.
3. SQL Server picks a victim and rolls it back with error 1205. The other commits (total $900).
4. The victim retries: SUM = 900, and 900 + 250 > 1,000, so it **refuses**.

Serializable isn't free. You pay with blocking and the occasional retry, and every write path needs a retry loop. In return you stop reasoning about anomalies one by one.

```
for attempt in range(5):
    try:
        run_transaction()
        break
    except DeadlockVictim:   # SQL Server error 1205
        continue             # retry, with backoff and jitter
```

Same retry rules as in the retries notes: a few attempts, backoff, jitter.

Compare with the others:

- **SQL Server snapshot isolation is not serializable.** It stops lost updates (error 3960) but still allows write skew.
- **PostgreSQL serializable (SSI)** doesn't block. It lets both run, spots a pattern no serial order could explain (like our two approvals) and aborts one with a serialization failure. It needs the same retry loop.

Deadlocks (headphones and charger)

Locks can form a loop. Say an order holds two products: the headphones (product id 7) and a charger (product id 8).

1. A locks the headphones (7), then wants the charger (8).
2. B locks the charger (8), then wants the headphones (7).

Each holds what the other needs, so both wait forever.

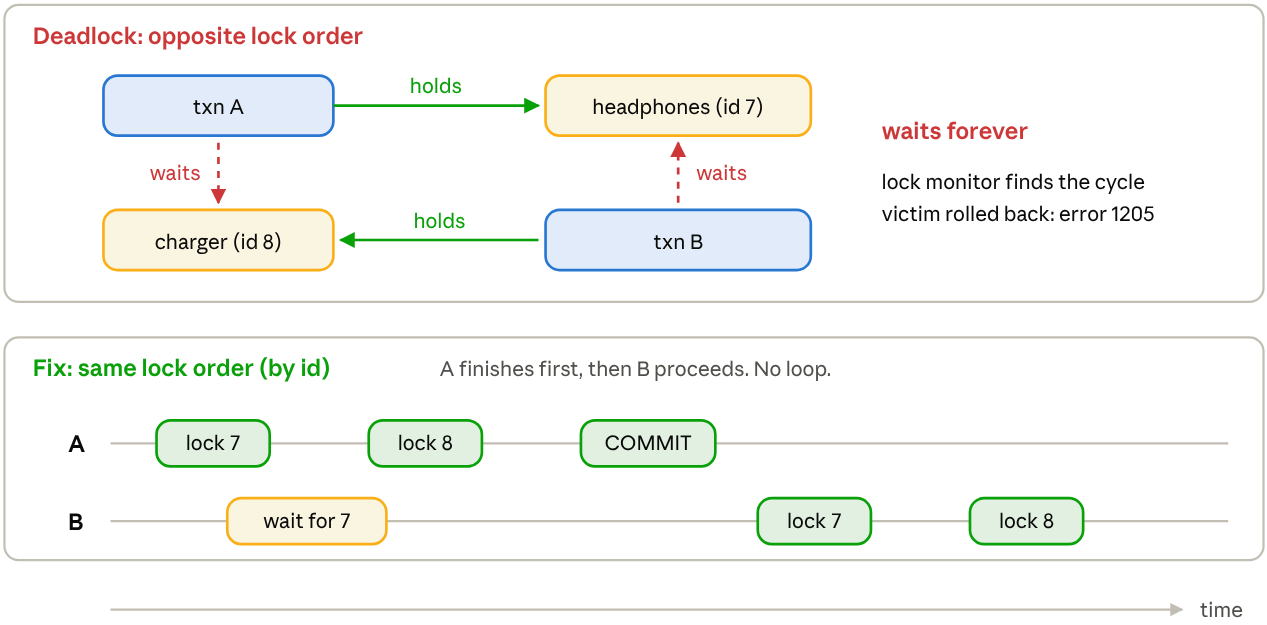
How SQL Server resolves it: a lock monitor looks for a cycle (by default every 5 seconds, and more often once deadlocks start happening). It picks a victim, usually the transaction cheapest to roll back, or the lower `SET DEADLOCK_PRIORITY`. The victim is rolled back with error 1205 and the other transaction finishes. PostgreSQL does the same after a lock wait of 1 s (`deadlock_timeout`).

The victim must retry, with the same retry loop as above.

The cure is in your code:

1. **Lock rows in a consistent order**, say by id. A and B both lock 7 first, then 8. B waits for 7 while A finishes, so there is no loop.
2. **Keep transactions short**, so locks are held for milliseconds, not seconds.

Opposite lock order deadlocks; the same lock order does not



deadlock and the consistent-order fix

Flash sale: selling the last units

A brand runs a flash sale on one product, or several. It is the headphones problem again, now with 500 units and 200K+ buyers. If one pair is left and two shoppers want it, one must get it and the other must not.

The rule to protect: **sold + held ≤ stock.**

Why read-then-write breaks it

```
s = read(stock)
if s > 0:
    write(stock, s - 1)
```

Both shoppers read 1, both decide there is enough, both write 0, and both get a confirmation. **2 sales, 1 pair.** We charge two people and deliver one, which is wrong. This is the lost update from above, on `sneaker-7`.

Fix 1: lock the row (correct, but hot)

```
BEGIN TRAN;
SELECT stock FROM inventory WITH (UPDLOCK, ROWLOCK) WHERE item = 'sneaker-7';
-- check, decrement
COMMIT;
```

It is correct, but everyone queues on one row. If each transaction holds the lock for about 2 ms, the row serves at most 1 / 2 ms = **500 reservations per second, no matter how many servers we add.** Worse, while they queue they hold database connections, so the pool fills up and the rest of the store feels it.

Fix 2: atomic conditional decrement

```
UPDATE inventory
SET stock = stock - 1
WHERE item = 'sneaker-7' AND stock > 0;
-- 1 row: you got a pair
-- 0 rows: sold out
```

In Redis the same idea is `DECR`, or a Lua check-and-decrement. It is correct and fast, because the lock lasts only one statement. But it is **still one hot row**: every writer lines up on it, so it caps the throughput.

Reserve, claim, release

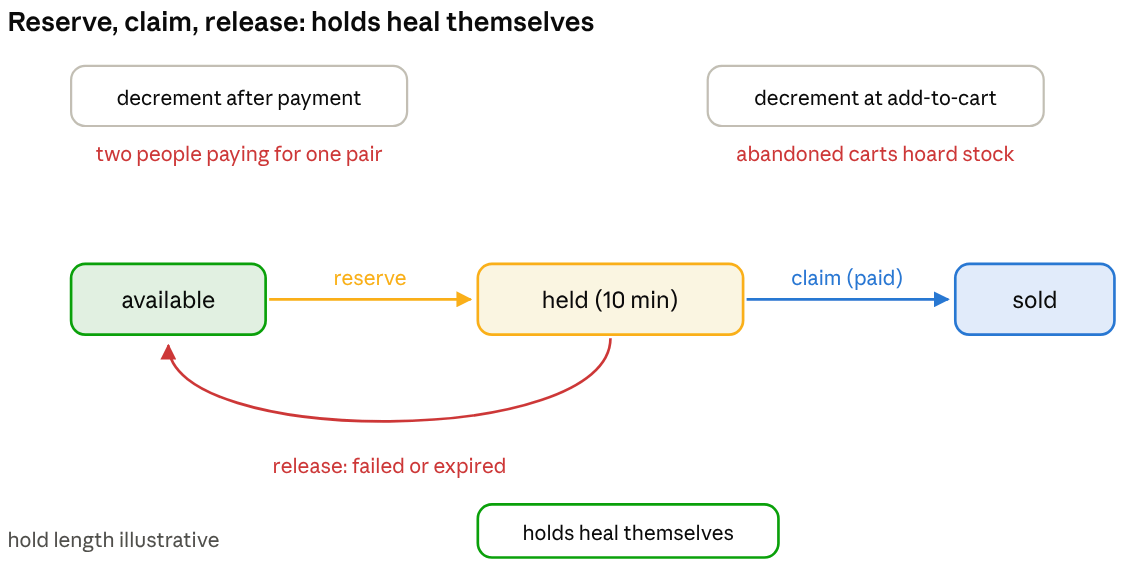
When should the stock go down? Both obvious answers fail:

- **Decrement after payment:** two people end up paying for one pair.
- **Decrement at add-to-cart:** abandoned carts hoard stock.

The way out is a **hold with a timer**. Payment is a foreign system, and like a payment timeout in the retries notes, we can't know in advance whether it will succeed. So we hold the unit for that shopper while the payment is attempted.

1. **Reserve:** the first shopper's request moves one unit from available to **held** for a limited time (10 minutes here, for illustration).
2. **Claim:** the payment succeeds, so the unit becomes **sold**.
3. **Release:** the payment fails or the timer runs out, so the unit goes back to **available** for the next shopper.

Holds heal themselves: nothing stays stuck if a shopper walks away.



reserve, claim, release

Splitting the hot row (Shopify, 2026)

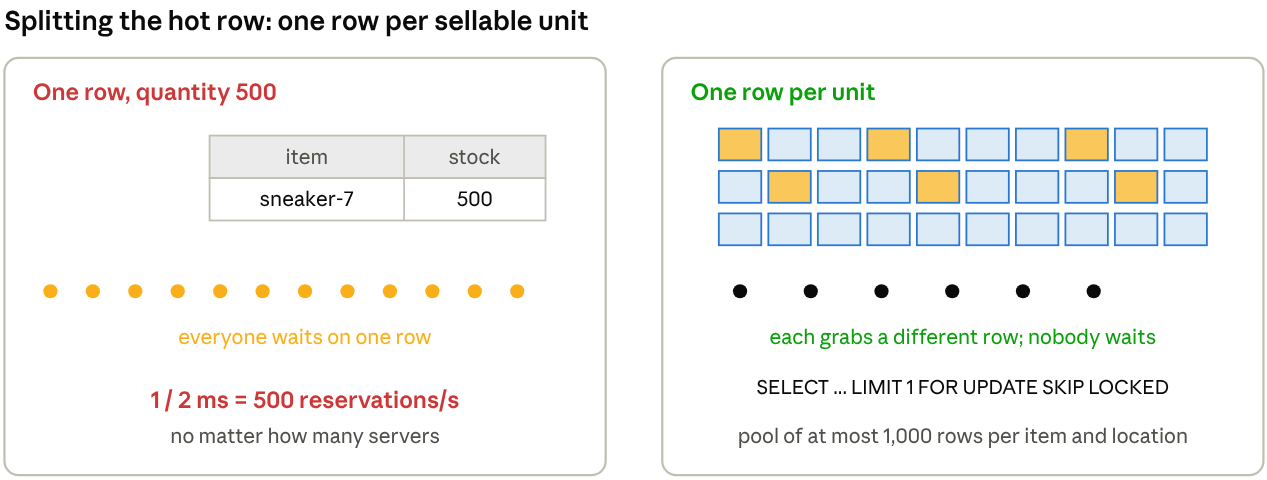
Even with holds, an atomic decrement leaves one hot row. Shopify first tried to split the work across two systems: **Redis for the holds, a MySQL ledger for the sales.** The problem is that a claim then has to write to both, and **two writes are not atomic.** If either system fails, the numbers drift apart.

The fix was to split the hot row itself. Instead of one row holding quantity 500, keep **one row per sellable unit.**

- Each shopper grabs a **different** row and skips the locked ones, so nobody waits: `SELECT ... LIMIT 1 FOR UPDATE SKIP LOCKED` (MySQL).
- The pool is capped at about 1,000 rows per item and location, and is refilled from the ledger.
- It stays in the **same database**, so these are real transactions. Shopify reports writer CPU under 50% during flash sales (Shopify Engineering, May 2026).

In SQL Server the same idea is `READPAST`, which skips rows that other transactions have locked:

```
BEGIN TRAN;
SELECT TOP (1) unit_id
FROM stock_units WITH (UPDLOCK, READPAST, ROWLOCK)
WHERE item = 'sneaker-7' AND status = 'available'
ORDER BY unit_id;
-- mark that unit as held, then COMMIT
```

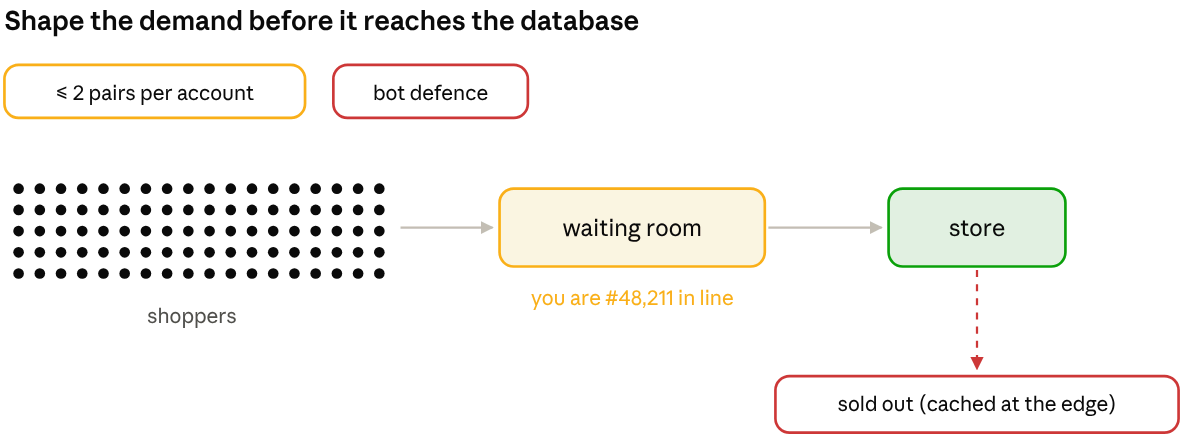


one hot row vs one row per unit

Shape the demand

Stock is 500 and there are 200K+ buyers, so even one row per unit can't take them all at once. Reduce the demand before it reaches the database.

- **Waiting room:** shoppers get a place in line ("you are #48,211") and are let into the store at a pace the system can handle.
- **Per-account limit:** at most 1 or 2 pairs per account.
- **Bot defence:** bots will be most of the traffic.
- **Sold out, cached at the edge:** when the stock hits zero, set a sold-out flag in Redis. Everyone in the waiting room gets "sold out" without touching the database.
- **Idempotency key per checkout:** a shopper who double clicks, or retries on a slow network, must not reserve twice (see the Retries, Timeout and Idempotency notes).



Every checkout carries an idempotency key, so a double click or a retry doesn't reserve twice.

waiting room, limits, sold out at the edge

Keep the numbers honest

Stock must never go below zero, and when everything is sold it must be exactly zero. Splitting rows and shaping demand are only safe if the numbers stay right:

- **One source of truth:** the ledger decides. The Redis flag is a shortcut, not the truth.
- **`CHECK (stock >= 0)` in the database**, as the last line of defence:

```
ALTER TABLE inventory ADD CONSTRAINT ck_stock_nonnegative CHECK (stock >= 0);
```

- **Same lock order on every path.** Shopify hit deadlocks from opposite orders. Explained in the next section.
- **Reconcile** holds, sales and stock regularly.

In short: split the hot row, shape the demand, and keep a single ledger as the source of truth.

Same lock order on every path

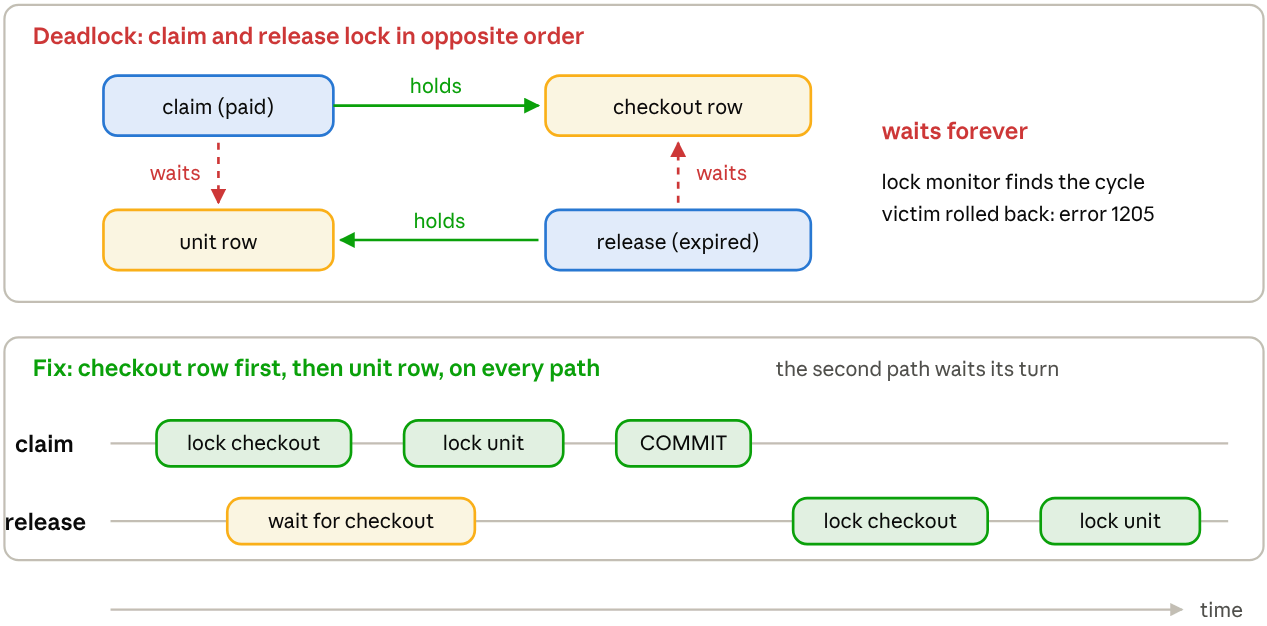
Locks themselves are fine. Deadlocks come from taking them in a **different order** in different code paths. Shopify hit exactly this: deadlocks from opposite orders. Splitting the hot row spreads the locks out, but it doesn't remove the rule.

A worked example (illustrative, your own paths will differ). Two paths both touch the checkout row and the unit row:

- **Claim** (payment succeeded): locks the checkout row, then the unit row, and marks the unit sold.
- **Release** (hold expired): locks the unit row, then the checkout row, and marks the unit available.

They can run at the same instant, when a payment lands just as the hold expires. Claim holds the checkout row and wants the unit row. Release holds the unit row and wants the checkout row. Each waits for the other: a **deadlock**.

Claim and release deadlock in opposite order; the same order does not



claim vs release: opposite order and the fix

The deadlock, step by step

Setup: Asha reserved sneaker unit 42 for checkout 812. Her hold expires at 10:10. At 10:10:00 her payment succeeds, and the expiry sweeper fires at the same moment. Two transactions now touch the same two rows (checkout 812 and unit 42), but in opposite order.

Step	Claim (payment succeeded)	Release (hold expired)	Result
1	`UPDATE checkout SET status = 'paid' WHERE id = 812`		Claim holds **checkout 812**

Step	Claim (payment succeeded)	Release (hold expired)	Result
2		`UPDATE unit SET status = 'available' WHERE id = 42`	Release holds **unit 42**
3	`UPDATE unit SET status = 'sold' WHERE id = 42`		Claim needs unit 42, held by Release, so it **waits**
4		`UPDATE checkout SET status = 'expired' WHERE id = 812`	Release needs checkout 812, held by Claim, so it **waits**

Claim waits for Release and Release waits for Claim. Neither can commit, so neither can ever let go. SQL Server finds the cycle, kills one of them with error 1205 and the other continues.

This is worse than it looks. If Claim is the victim, Asha has **already paid**, but her claim was rolled back and the sweeper has just released her unit. So we don't want to depend on the victim being retried. We want the deadlock not to happen.

The fix is to pick **one order for every path**, say **checkout row first, then unit row**. Reserve creates the checkout row first (a new row, so no contention) and then picks a unit with `SKIP LOCKED`. Claim and release also lock the checkout row first. Now whichever path arrives second just waits for the first to finish. No loop, no deadlock.

`SKIP LOCKED` helps here, since reserve never waits on a locked unit. But claim and release lock a **specific** unit and can wait, so the order still matters.

The fix, step by step

Same setup, but now **both paths lock the checkout row first, then the unit row.** Each also checks the checkout is still `held` once it has the lock.

```
-- Claim
BEGIN TRAN;
UPDATE checkout SET status = 'paid' WHERE id = 812 AND status = 'held';   --
lock 1
IF @@ROWCOUNT = 1
    UPDATE unit SET status = 'sold' WHERE id = 42;                        --
lock 2
COMMIT;

-- Release
BEGIN TRAN;
UPDATE checkout SET status = 'expired' WHERE id = 812 AND status = 'held'; --
lock 1
IF @@ROWCOUNT = 1
    UPDATE unit SET status = 'available' WHERE id = 42;                    -
- lock 2
COMMIT;
```

Step	Claim (payment succeeded)	Release (hold expired)	Result
1	locks **checkout 812**, sets `paid`		Claim holds checkout 812
2		tries to lock checkout 812	Release **waits**, and holds nothing yet
3	locks **unit 42**, sets `sold`, `COMMIT`		Locks released, unit 42 is sold
4		gets checkout 812, but status is no longer `held`, so `@@ROWCOUNT = 0`	Release does nothing and commits

Outcome: one winner, no cycle, no victim. The waiting path never holds a lock the other one needs, because both ask for the checkout row first.

If Release had arrived first, the roles swap. Release locks the checkout, marks it `expired` and frees the unit. Claim then finds the checkout is no longer `held`, so `@@ROWCOUNT = 0`. The payment arrived too late, so the app refunds it or tries to reserve again. That is a clean, expected outcome, not a rolled-back victim.

How to apply it:

1. **Write the order down**, for example checkout row, then unit row, and items by id.
2. **Every path follows it:** reserve, claim, release, the expiry sweeper, the pool refill and the reconcile job. One path breaking the order is enough to deadlock.
3. **Several rows of the same kind:** lock them in id order (`ORDER BY`).
4. **Never go backwards.** If you need a lock that belongs earlier in the order, roll back and start again.
5. **Keep transactions short.**
6. **Keep the safety net:** retry the deadlock victim (error 1205). In SQL Server the `system_health` extended event records deadlock graphs, so you can see which paths collided.

Putting it together

The flash sale design is a few pieces that each solve one problem. Two clarifications first:

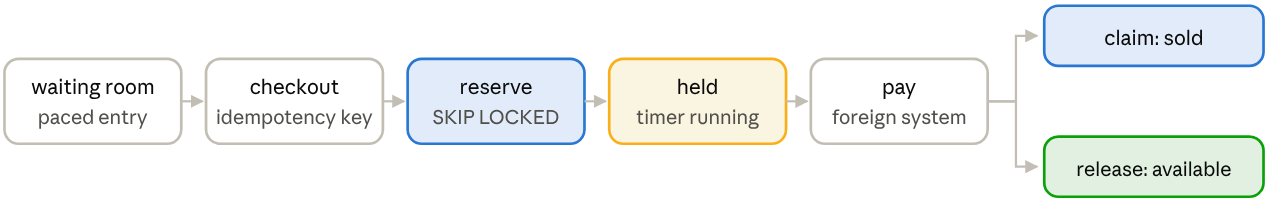
- The row split doesn't remove locks, it **spreads** them. Each shopper still locks a row, but a different one, and `SKIP LOCKED` lets a request move past rows that are already locked.
- The **waiting room** is not what skips locked rows. The database does that. The waiting room controls how many shoppers reach the database at all, because 200K+ buyers is more than the system can take even with one row per unit.

Piece	What it does	Problem it solves
One row per unit + `SKIP LOCKED`	Each shopper locks a different unit and skips locked ones	One hot row serializes everyone (about 500 per second)
Waiting room, limits, bot defence, sold-out flag	Admits only as many shoppers as the system can take	200K+ buyers against 500 units
Idempotency key per checkout	Same key on every retry gives one reservation	Double clicks and slow-network retries reserving twice

Piece	What it does	Problem it solves
Reserve, claim, release	Hold a unit with a timer, claim on payment, release otherwise	Payment is a foreign system and may fail
One ledger, `CHECK`, same lock order, reconcile	Keeps the numbers honest	Oversell, drift and deadlocks

From your slides, Shopify is named for splitting the hot row into one row per unit in the same database, and for the deadlock lesson. The waiting room, idempotency key and holds are the supporting pieces of the same flash sale design.

One shopper, end to end



A retry or double click carries the same idempotency key, so it can't reserve a second unit.

Reserve, claim and release all lock the checkout row first, then the unit row.

Paid within the hold: claim. Payment fails or the timer runs out: release.

one shopper, end to end

Recap

1. Isolation is the I in ACID: each transaction should look like it ran alone. That ideal is serializable.
2. Databases let transactions overlap to keep throughput up, so we choose how much interference to allow.
3. Three anomalies: dirty read, non-repeatable read, phantom. Each level prevents more of them.
4. Level names are the same across databases but the behavior is not. Check your engine. Defaults: PostgreSQL read committed, MySQL repeatable read, SQL Server read committed.
5. Read-modify-write loses updates (the headphones). Fix it with an atomic guarded UPDATE, `UPDLOCK`, or a version check.

6. Write skew (the budget approval) beats row locks and snapshots. Lock the premise, or use serializable and retry.
7. Consistent lock order plus short transactions prevent deadlocks. Always retry the victim (error 1205).
8. Flash sale rule: sold + held ⩽ stock. Read-then-write breaks it.
9. An atomic conditional decrement is correct, but one hot row caps the throughput.
10. Reserve with a timer, then claim or release.
11. Split the hot row (one row per unit, `SKIP LOCKED` / `READPAST`), shape the demand, keep one ledger as the source of truth, and take locks in the same order on every path.